InCollege Project: Week 6 Deliverable - Job/Internship Postings - Basic Functionality

Objective: This week, your team will implement the foundational capability for users to post new job or internship listings on the InCollege platform. This involves capturing essential job details and ensuring they are saved persistently, all while adhering to the file-based input and output requirements.

Focus Areas:

1. **Job Posting Option:**
    - The "Job search/internship" option, which was previously an "under construction" message, will now become functional.
    - When a user selects this option, they should be presented with a choice to either "Post a Job/Internship" or "Browse Jobs/Internships" (the latter will remain "under construction" for this week).
    - The user will be presented with an option to return to the top level menu.
2. **Capturing Job Details:**
    - If the user chooses to "Post a Job/Internship," the system should prompt them to enter the following details for the listing:
        - **Job Title:** (Required, e.g., "Software Intern," "Data Analyst")
        - **Description:** (Required, a brief text description of the role)
        - **Employer:** (Required, the name of the company or organization)
        - **Location:** (Required, e.g., "Remote," "New York, NY")
        - **Salary (Optional):** (e.g., "$50,000/year," "$25/hour")
3. **Posting Persistence:**
    - All job and internship listings created by users must be saved persistently. This means they should be stored in a way that allows them to be retrieved even after the application is closed and reopened.
    - Each posting should ideally be associated with the user who created it, though for this week, simply storing the listings is the primary goal.
4. **Saving Jobs That You Are Interested In**
    - When a student is interested in a job, but does not want to apply for it right now, they can mark it as being "saved".
    - The student will be able to generate a list of the jobs that they have marked as being saved.
    - They will also be able to unmark a job as being saved if they change their mind about it.
    - The list of saved jobs and applied for jobs will be retained and can be displayed the next time that the student logs in.

5. **I/O Requirements:**
    - **Input:** All user input (e.g., menu selections, job details) will be read from a predefined input file.
    - **Output Display:** All program output (e.g., prompts, confirmation messages) must be displayed on the screen (standard output).
    - **Output Preservation:** The exact same output displayed on the screen must also be written to a separate output file for testing and record-keeping purposes.

COBOL Implementation Details (For Programmers):

- Your code is getting large. It is suggested that the Jobs code and Browse Jobs code be placed into separate Copybooks: JOBS_SRC (Jobs.cob) and BROWSEJOBS_SRC (BrowseJobs.cob).
- **Job Posting Data Structure:** Define a new COBOL data structure (e.g., a new sequential file) specifically for storing job and internship listings. This structure will need fields for each of the details outlined above (Title, Description, Employer, Location, Salary). Consider how to uniquely identify each job posting (e.g., a simple sequential ID).
- **File I/O for Job Postings:** Implement COBOL file operations to write new job postings to your persistent storage.
- **Input Handling:** Develop COBOL modules to prompt the user for each piece of job information and store it correctly. Ensure all these inputs are read from your designated input file.
- **Output Handling:** Ensure all program output, including prompts for job details and confirmation messages, is displayed on the screen and *identically* written to your dedicated output file.
- **Menu Integration:** Integrate the new "Post a Job/Internship" option into the "Job search/internship" menu path. The "Browse Jobs/Internships" option should still display an "under construction" message.

Testing Responsibilities (For Testers):

- **Test Case Development:** Create comprehensive test cases in Jira for all Week 6 functionalities, including:
    - **Positive Test Cases:** Scenarios for successfully posting a job with all required fields, and with optional salary included.
    - **Negative Test Cases:** Scenarios for attempting to post a job with missing required fields, or invalid input where applicable (e.g., very long strings if a character limit is implemented implicitly by COBOL PIC clauses).
    - **Persistence Testing:** Verify that posted jobs are saved and are still present after the program is restarted (requires coordination with programmers to manually check data file content).
- **Test Execution:** Execute all developed test cases using the specified input file.
- **Bug Reporting:** For every issue or discrepancy found, create a detailed bug ticket in Jira. Include steps to reproduce, actual results, and expected results.

- **Output Verification:** Meticulously compare the program's console output against the generated output file to ensure they are absolutely identical for all job posting scenarios.
- **Collaboration:** Work closely with the programmers to help them understand and reproduce bugs.

Jira Requirements (For Scrum Master, Programmers, & Testers):

Your team's Jira board for Week 6 should include:

- **Epic #4: Job Board Functionality:** Define this new epic for all job board related features.
- **User Stories for Posting Jobs:**
  - "As a logged-in user, I want to post a new job or internship listing so I can recruit for my company/organization."
  - "As a user posting a job, I want to enter a job title, description, employer, and location."
  - "As a user posting a job, I want the option to include a salary for the listing."
  - "As a user, I want my posted job listings to be saved persistently."
  - "As a user, I want the 'Job search/internship' menu option to allow me to post jobs."
- **New User Story (for Testing):** "As a tester, I want the program to read all user inputs for job posting from a file so I can automate testing."
- **New User Story (for Testing):** "As a tester, I want the program to write all screen output related to job posting to a file so I can easily verify results."
- **Tasks:** Break down each User Story into granular tasks that individual team members can work on. (e.g., *Programmers:* "Define COBOL file structure for job postings," "Implement COBOL module to capture job details input," "Develop COBOL routine to write job data to file," "Integrate job posting option into main menu flow," "Update I/O for job posting features to use file input/output consistently." *Testers:* "Develop test cases for job posting," "Execute job posting tests with valid/invalid data," "Log bugs related to job posting," "Verify I/O consistency for job posting features").
- **Bug Tickets:** Log any issues found during development and testing.

GitHub Requirements:

- **New Modules/Files:** Commit any new COBOL modules or updated existing files for handling job postings.
- **Branching:** Continue to follow your team's established branching strategy for new features.
- **Regular Commits:** Ensure consistent, descriptive commits throughout the week. Testers should commit their test files.
- **README.md Update:** Update your README.md to reflect the new functionality for posting jobs, explicitly detailing how to prepare input for these features and where to find the corresponding output.

Deliverables for End of Week 6:

1. **Roles.txt**: List of team members and the roles that they played this week.
2. **InCollege.cob: Working COBOL Program:** A console-based COBOL application that allows logged-in users to:
    a. Navigate to a "Job search/internship" menu.
    b. Select an option to "Post a Job/Internship."
    c. Enter all required and optional job details.
    d. Persistently save these job listings.
    e. The "Browse Jobs/Internships" option should display an "under construction" message.
    f. This must seamlessly integrate with all previous weeks' functionality (login, profile, connections). All inputs must be read from a file, all outputs displayed on the screen, and the exact same outputs written to a separate file.
3. **InCollege-Input.txt: Sample Input File:** A sample text file demonstrating the format of input your program expects for Week 6's functionality (e.g., menu choices leading to job posting, job detail inputs).
4. **InCollege-Output.txt:** A sample text file showing the expected output for a typical run of your program, demonstrating a user posting a job.

```
--- SAMPLE_OUTPUT_WEEK6.TXT ---
Welcome to InCollege!
1. Log In
2. Create New Account
Enter your choice:
Please enter your username:
Please enter your password:
You have successfully logged in.
Welcome, MyUser!
1. Search for a job
2. Find someone you know
3. Learn a new skill
4. View My Pending Connection Requests
5. View My Network
Enter your choice:
--- Job Search/Internship Menu ---
1. Post a Job/Internship
2. Browse Jobs/Internships
3. Back to Main Menu
Enter your choice:
--- Post a New Job/Internship ---
Enter Job Title:
Enter Description (max 200 chars):
Enter Employer Name:
Enter Location:
Enter Salary (optional, enter 'NONE' to skip):
Job posted successfully!
---------------------------------
1. Post a Job/Internship
```

2. Browse Jobs/Internships
3. Back to Main Menu
Enter your choice:
Browse Jobs/Internships is under construction.
1. Post a Job/Internship
2. Browse Jobs/Internships
3. Back to Main Menu
Enter your choice:
--- END_OF_PROGRAM_EXECUTION ---

5. **Epic6-Storyx-Test-Input.zip: Test Input Files:** A set of test input files used by the testers, covering positive, negative, and edge cases for each of this week's stories.
6. **Epic6-Storyx-Test-Output.zip: Actual Test Output Files:** The exact output generated by running your program with the Epic6-Storyx-Input input Files, submitted for review.
7. **Jira.jpg: Updated Jira Board:** All relevant User Stories, tasks, and bugs (with their status) for Week 3 should be updated in Jira.
8. **Jira:** Two Burndown charts. The first created on Monday and the second created when the Sprint is complete.
9. **GitHub.jpg:** Go to the repository's main page. Click the "Commits" link (next to the green "Code" button). Show a chronological list of all commits with messages, authors, and timestamps.

Your testers will be crucial this week in verifying that all job details are captured and saved correctly, and that the data persists across application restarts. They should also ensure the navigation flows correctly to and from the job posting feature, all while validating the consistency of screen and file output. The scrum master will, as always, be key to managing the sprint and facilitating the team's progress.